Experiment No. 1

Title: Study of Raspberry-Pi, Beagle board, Arduino, and different operating systems for Raspberry Pi/Arduino.

Problem Statement: To study of Raspberry-Pi, Beagle board, Arduino, and different operating systems for Raspberry Pi/Beagle board/Arduino.

ELO-1: Students will be able to understand various IoT Interfacing boards.

Objective:

1. To understand the Rasberry-Pi board.
2. To understand the Ardino board.
3. To understand various operating systems required for Rasberry-pi/Arduino board.

S/W Packages and H/W Used:

Arduino IDE, Arduino Board, Raspberry Pi Board

Compare and Contrast Existing Hardware Tools:

Nome	Arduino Uno	Raspberry Pi
Modelo	R3	Modelo B
Preço	R$75,00	R$250,00*
Tamanho	2,95"x2,10"	3,37"x2,125"
Processador	Atmega328	ARM11
Velocidade de Clock	16MHz	700Mhz
RAM	2KB	256MB
Flash	32KB	Cartão SD
EEPROM	1KB	-
Tensão de alimentação	7V a 12V	5V
Corrente mínima	42mA	700mA
pinos GPIO	14	8
Entradas analógicas	6	-
PWM	6	-
TWI/I2C	2	1
SPI	1	1
UART	1	1
Interface de Desenvolvimento	Arduino	IDLE, Scratch, Squeak/Linux, Python
Ethernet	Apenas com uso de shield	10/100
USB	-	2 USB 2.0
Saída de vídeo	Pode-se montar	HDMI, RCA
Saída de áudio	-	Jack

Theory:

Arduino:

An Arduino board consists of an Atmel 8-bit AVR microcontroller with complementary components that facilitate programming and incorporation into other circuits. It's an open-source physical computing platform based on a simple microcontroller board, and a development environment for writing software for the board.



Figure 1.1: Arduino UNO

Specification of Arduino Uno:

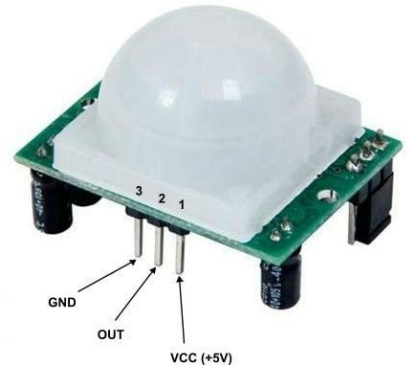
1.	Microcontroller	:ATmega328P
2.	Operating Voltage	:5V
3.	Input Voltage (recommended)	:7-12V
4.	Input Voltage (limit)	:6-20V
5.	Digital I/O Pins	:14 (of which 6 provide PWM output)
6.	Analog Input Pins	6
7.	DC Current per I/O Pin	:20 mA
8.	DC Current for 3.3V Pin	:50 mA
9.	Flash Memory	:32 KB (ATmega328P) of which 0.5 KB used
10.	SRAM	:2 KB (ATmega328P)
11.	EEPROM	:1 KB (ATmega328P)
12.	Clock Speed	:16 MHz

Basic Sensors Used in Arduino Practicals

Arduino is a microcontroller board used to read data from sensors, process the input, and control output devices such as LEDs, buzzers, motors, and displays.

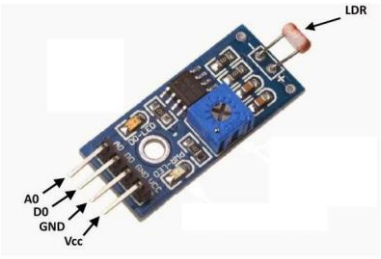
A sensor detects a physical condition such as motion, light, temperature, gas concentration, or electric current and converts it into an electrical signal that Arduino can read.

1. Overview of basic sensors



1. PIR Sensor (Motion Sensor)

- Full form: Passive Infrared Sensor.
- Detects: Motion of people or animals by sensing changes in infrared radiation.
- Pins: VCC, OUT, GND.
- Arduino application: Automatic lights, security alarms, and occupancy detection.
- Output: Usually digital HIGH when motion is detected and LOW otherwise.



2. LDR (Light Sensor)

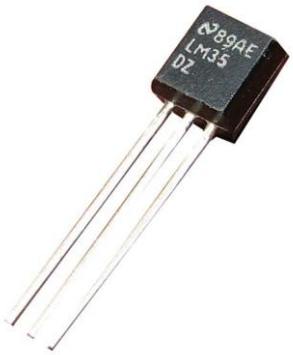
- Full form: Light Dependent Resistor.
- Detects: Light intensity.
- Pins: A bare LDR has two leads; a common module may have VCC, GND, AO and DO.

- Arduino application: Automatic streetlights, light monitoring, and brightness control.
- Output: Resistance changes with light. A module may provide analog and digital outputs.



3. Gas Sensor (MQ Series)

- Detects: Certain combustible gases and, depending on the model, smoke or other gases.
- Pins: Common modules have VCC, GND, AO and DO.
- Arduino application: Gas leakage demonstrations, smoke detection, and laboratory monitoring.
- Output: Analog gas-response signal and/or digital threshold signal.
- Example: MQ-2 sensor.



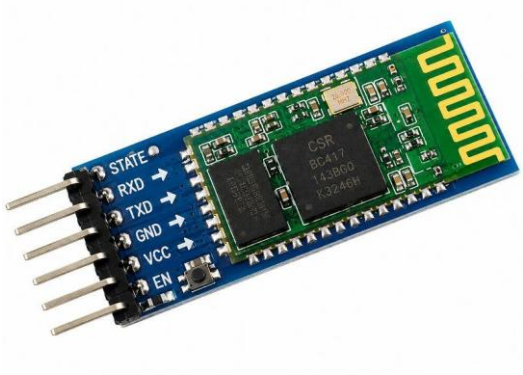
4. Temperature Sensor

- Detects: Temperature of the surrounding environment or the measured object, depending on the sensor.
- Examples: LM35, DHT11, DHT22, DS18B20.
- Arduino application: Temperature monitoring, fan control, and environmental monitoring.
- Output: Analog voltage for LM35; digital data for sensors such as DHT11 and DS18B20.



5. Current Sensor

- Detects: Electric current flowing through a circuit.
- Examples: ACS712, INA219.
- Arduino application: Energy monitoring, current measurement, and overload monitoring.
- Output: ACS712 produces an analog voltage; INA219 provides digital measurements over I²C.



6. Bluetooth Module (Wireless Communication)

- Example: HC-05.
- Purpose: Sends and receives data wirelessly between Arduino and a compatible phone or computer.
- Pins: VCC, GND, TXD, RXD; some boards also expose KEY/EN and STATE.
- Arduino application: Wireless LED control, password-based access systems, and remote sensor monitoring.
- Output: Serial data, such as commands or sensor readings.

Raspberry Pi :

The Raspberry Pi is a series of small single-board computers developed in the United Kingdom by the Raspberry Pi Foundation to promote the teaching of basic computer science in schools and in developing countries.

The original model became far more popular than anticipated, selling outside of its target market for uses such as robotics.

Over 5 million Raspberry Pis have been sold before February 2015, making it the best-selling British computer. By November 2016 they had sold 11 million units



Figure 1.2: Raspberry Pi 3 B+

Features:

- All models feature a Broadcom system on a chip (SoC), which includes an ARM compatible central processing unit (CPU) and an on-chip graphics processing unit (GPU, a VideoCore IV).
- CPU speed ranges from 700 MHz to 1.5 GHz for the Pi 4 and on board memory range from 256 MB to 4 GB RAM.
- Secure Digital (SD) cards are used to store the operating system and program memory in either the SDHC or Micro SDHC sizes.
- Most boards have between one and four USB slots, HDMI and composite video output, and a 3.5 mm phone jack for audio.

- Lower level output is provided by a number of GPIO pins which support common protocols like I²C.
- The B-models have an 8P8C Ethernet port and the Pi 3 and Pi Zero W have on board Wi-Fi 802.11n and Bluetooth.

Conclusion:

Experiment No. 2

Title:

Write a program for sending alert messages to the user for controlling and interacting with your environment.

Aim:

To write and implement an Arduino program that allows the user to control an LED and monitor environmental temperature through a Bluetooth-connected mobile application. The program should display instructions, received commands, temperature readings, and alert messages on the Arduino IDE Serial Monitor.

Components Used:

Sr. N	Component	Quantit
1	Arduino UNO	1
2	HC-05 Bluetooth module	1
3	LM35 temperature sensor	1
4	LED	1
5	220 Ω resistor	1
6	Breadboard and connecting wires	As requir
7	USB cable	1
8	Android phone with Arduino Bluetooth Controller a	1

Theory:

Bluetooth is a wireless communication technology used to exchange data between electronic devices. In this experiment, the HC-05 Bluetooth module connects the Arduino UNO to an Android mobile phone.

The system works as follows:

1. The user sends commands using the Arduino Bluetooth Controller app.

2. The HC-05 receives the commands and transfers them to the Arduino.
3. The Arduino switches the LED ON or OFF according to the received command.
4. The LM35 sensor measures the surrounding temperature and provides an analog voltage to Arduino pin A0.
5. The Arduino calculates the temperature and sends the readings and alert messages to the mobile app and Serial Monitor.

The program generates a high-temperature alert when the measured temperature exceeds 35°C.

Procedure:

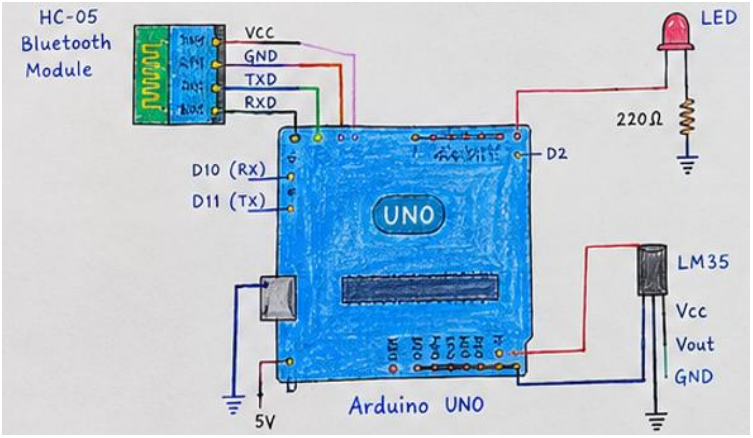
1. Connect the LED, resistor, HC-05 Bluetooth module and LM35 sensor to the Arduino UNO according to the connection table.
2. Connect the Arduino UNO to the computer using a USB cable.
3. Open Arduino IDE, select the correct board (Arduino UNO) and COM port.
4. Copy the program, click Verify, and then click Upload.
5. Open Tools → Serial Monitor and select a baud rate of 9600.
6. Pair the Android phone with the HC-05 module. The default pairing PIN is often 1234 or 0000, depending on the module.
7. Open the Arduino Bluetooth Controller app and connect to the paired HC-05 device.
8. Send L to switch ON the LED and O to switch it OFF. Observe the messages on the Serial Monitor.
9. Send T to read the temperature and observe the temperature value and alert status.
10. Send S to check the LED status and confirm that the system is ready.

Connection table

Component	Pin	Arduino connection
HC-05	VCC	5V (check module breakout specifications)
HC-05	GND	GND
HC-05	TXD	D10 (Arduino software RX)
HC-05	RXD	D11 (Arduino software TX), through a suitable voltage divid

LM35	Vs / VC(	5V
LM35	Vout	A0
LM35	GND	GND
LED	Anode (+	D2 through 220 Ω resistor
LED	Cathode (	GND

Circuit Diagram:



Program:

```
#include <SoftwareSerial.h>
// HC-05: Arduino RX = D10, TX = D11
SoftwareSerial BT(10, 11);

const int ledPin = 2;
const int tempPin = A0;
const float alertTemp = 35.0;

void setup()
{
  pinMode(ledPin, OUTPUT);

  Serial.begin(9600);
  BT.begin(9600);

  Serial.println("=== Bluetooth Environment Control ===");
  Serial.println("Commands from mobile app:");
  Serial.println("L = LED ON");
  Serial.println("O = LED OFF");
  Serial.println("T = Read Temperature");
  Serial.println("S = System Status");
  Serial.println("-------------------------------------");
  Serial.println("System Ready!");
}

void loop()
{
  if (BT.available())
  {
    char cmd = BT.read();
```

```
Serial.print("Command received: ");
Serial.println(cmd);

if (cmd == 'L' || cmd == 'l')
{
  digitalWrite(ledPin, HIGH);
  Serial.println("LED turned ON");
  BT.println("LED turned ON");
}
else if (cmd == 'O' || cmd == 'o')
{
  digitalWrite(ledPin, LOW);
  Serial.println("LED turned OFF");
  BT.println("LED turned OFF");
}
else if (cmd == 'T' || cmd == 't')
{
  int adc = analogRead(tempPin);

  float voltage = adc * (5.0 / 1023.0);
  float tempC = voltage * 100.0;

  Serial.print("Temperature: ");
  Serial.print(tempC, 2);
  Serial.println(" C");

  BT.print("Temperature: ");
  BT.print(tempC, 2);
  BT.println(" C");
```

```
    if (tempC > alertTemp)
    {
      Serial.println("ALERT: High Temperature!");
      BT.println("ALERT: High Temperature!");
    }
    else
    {
      Serial.println("Temperature is Normal");
      BT.println("Temperature is Normal");
    }
  }
  else if (cmd == 'S' || cmd == 's')
  {
    Serial.print("LED Status: ");
    if (digitalRead(ledPin) == HIGH)
      Serial.println("ON");
    else
      Serial.println("OFF");

    Serial.println("System Ready");
    BT.println("System Ready");
  }
  else
  {
    Serial.println("Invalid command!");
    BT.println("Invalid command!");
  }
 }
}
```

Experiment No. 3

Title

Interfacing of PIR (Passive Infrared) Sensor with Arduino UNO for Motion Detection.

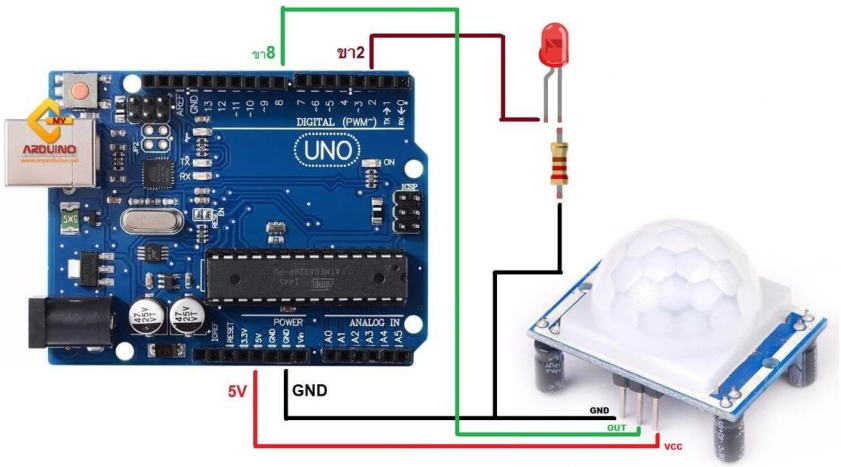
Aim

To interface a PIR sensor with an Arduino UNO and detect human motion. When motion is detected, the LED should turn ON and a message should be displayed on the Serial Monitor.

Components Used

Sr. No.	Component	Quantity
1	Arduino UNO	1
2	PIR motion sensor (HC-SR501)	1
3	LED	1
4	Resistor (220 Ω)	1
5	Breadboard	1
6	Jumper wires	As required
7	USB cable	1

Circuit Diagram



Connections

PIR Sensor / Component	Arduino UNO
VCC	5V
OUT	Digital pin 2
GND	GND
LED anode (+)	Digital pin 13 through 220 Ω resistor
LED cathode (−)	GND

Theory

A PIR (Passive Infrared) sensor detects motion by sensing changes in infrared radiation emitted by warm objects, such as humans and animals.

The HC-SR501 PIR sensor has three pins: VCC, OUT and GND. VCC supplies power, GND provides the ground connection, and OUT generates a digital signal indicating motion.

When a person moves within the sensor's detection area, the sensor output generally becomes HIGH. Arduino reads this signal and turns ON the LED. When the output becomes LOW, the LED turns OFF.

The Arduino also sends messages to the Serial Monitor, allowing the user to observe the motion detection status.

Applications include security systems, automatic lighting, home automation and occupancy monitoring.

Procedure

1. Make the circuit connections. Connect the PIR sensor's VCC to 5V, GND to GND and OUT to digital pin 2 of Arduino UNO.
2. Connect the LED. Use the built-in LED on pin 13 or connect an external LED through a 220 Ω resistor.
3. Open Arduino IDE. Connect the Arduino UNO to the computer using a USB cable.
4. Write the program. Enter the above code in the Arduino IDE.
5. Select the board and port. Choose Tools → Board → Arduino UNO and select the appropriate COM port.

6. Upload the program. Click the Upload button and wait for the upload to complete.
7. Open Serial Monitor. Select Tools → Serial Monitor and set the baud rate to 9600.
8. Test motion detection. Allow the PIR sensor to stabilize after powering it on. Then move your hand or walk in front of the sensor.
9. Observe the output. When motion is detected, the LED turns ON and the Serial Monitor displays "Motion Detected!". When no motion is detected, the LED turns OFF and "No Motion" is displayed.

Expected Output

Condition	PIR Output	LED Status	Serial Monitor
Motion detected	HIGH	ON	Motion Detected!
No motion detected	LOW	OFF	No Motion

Conclusion

The PIR sensor was successfully interfaced with the Arduino UNO for motion detection. The system detects changes in infrared radiation caused by moving objects, turns ON the LED when motion is detected and displays the corresponding status on the Serial Monitor. Thus, the experiment demonstrates a basic motion-detection system that can be used in security and home automation applications.

Program:

```
const int pirPin = 2;
const int ledPin = 13;

void setup() {
  pinMode(pirPin, INPUT);
  pinMode(ledPin, OUTPUT);

  Serial.begin(9600);
  Serial.println("PIR Sensor Initializing...");
}

void loop() {
  int motion = digitalRead(pirPin);

  if (motion == HIGH) {
    digitalWrite(ledPin, HIGH);
    Serial.println("Motion Detected!");
  }
  else {
    digitalWrite(ledPin, LOW);
    Serial.println("No Motion");
  }

  delay(500);
}
```

Experiment No. 4

Title: Design and Develop a User Interface for Monitoring and Controlling a CPS System Using Arduino

Aim

To design and develop a user interface for monitoring sensor data and controlling output devices in a Cyber-Physical System (CPS) using Arduino Uno and the Serial Monitor of the Arduino IDE.

Components Required

Sr. No.	Component	Quantity
1	Arduino Uno	1
2	LED	1
3	220 Ω resistor	1
4	LDR (light sensc	1
5	10 kΩ resistor	1
6	Breadboard	1
7	Jumper wires	As requir
8	USB cable	1
9	Arduino IDE	1

Theory

A Cyber-Physical System (CPS) integrates physical devices, sensors, actuators, computing systems, and software to monitor and control a real-world environment.

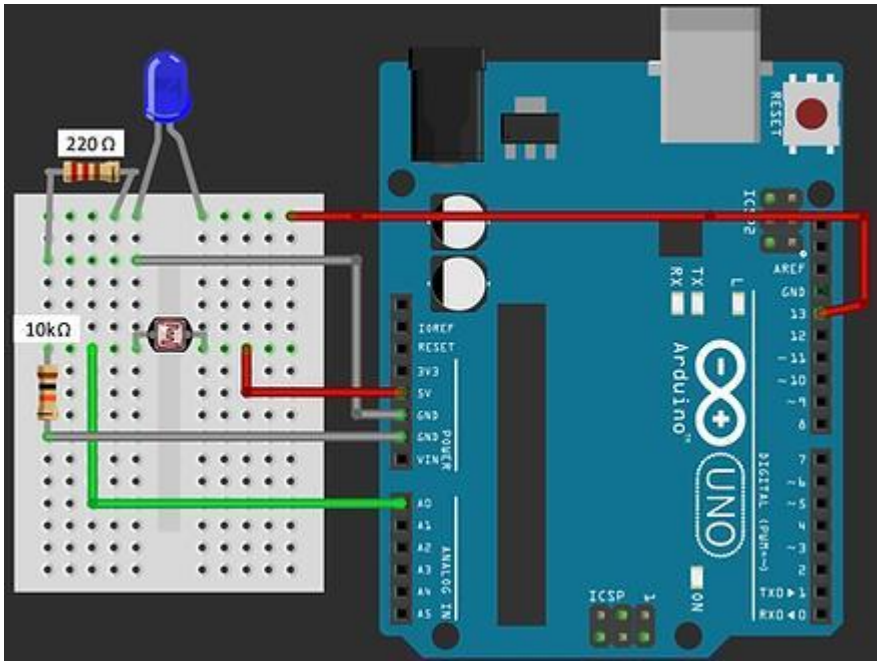
In this experiment:

- Sensor: An LDR measures the surrounding light intensity.
- Controller: Arduino Uno reads the sensor value and processes user commands.
- User interface: The Serial Monitor displays sensor readings and accepts commands from the user.
- Actuator: An LED represents a controlled physical device.

- Feedback: The Arduino displays the current sensor value and LED status.

The user can enter commands such as ON, OFF, and STATUS through the Serial Monitor to control the LED and monitor the system.

Circuit Connections



a

Connect the components as follows:

Component	Arduino connection
LDR first terminal	5V
LDR second terminal	A0 and one terminal of 10 kΩ resistor
10 kΩ resistor other terminal	GND
LED anode (long leg)	Digital pin 8 through 220 Ω resistor
LED cathode (short leg)	GND

The LDR and 10 kΩ resistor form a voltage divider that provides an analog light-level reading to pin A0.

Procedure

1. Connect the LDR, resistor, and LED to the Arduino Uno according to the circuit connections.
2. Open the Arduino IDE and select Tools → Board → Arduino Uno.
3. Select the correct COM port under Tools → Port.
4. Copy the program into the IDE, verify the code, and upload it to the Arduino.
5. Open Tools → Serial Monitor and set the baud rate to 9600.
6. Set the line ending to Newline or Both NL & CR.
7. Observe the continuously displayed light sensor values and LED status.
8. Enter ON to turn on the LED and OFF to turn it off.
9. Enter STATUS to display the current sensor reading and LED state.

Sample Output

The Serial Monitor displays output similar to the following:

```
==================================
 CPS MONITORING AND CONTROL UI
==================================
Commands:
ON     - Turn LED ON
OFF    - Turn LED OFF
STATUS - View system status
HELP   - Display commands
--------------------------------
Light Sensor Value: 620 | LED Status: OFF
Light Sensor Value: 618 | LED Status: OFF
ON
SUCCESS: LED turned ON
Light Sensor Value: 621 | LED Status: ON
OFF
SUCCESS: LED turned OFF
```

The sensor values shown are illustrative; actual readings depend on the light falling on the LDR.

Result

The user interface for monitoring and controlling a Cyber-Physical System was successfully developed using Arduino Uno. The system monitors light intensity through an LDR, displays sensor data on the Serial Monitor, and allows the user to control an LED using serial commands.

Applications

- Smart home lighting systems
- Industrial equipment monitoring
- Environmental monitoring
- Automated control systems
- Smart agriculture and greenhouse monitoring

Connection Table – Arduino CPS Monitoring and Control System

Sr. No.	Component	Pin/Terminal	Arduino Uno Connection
1	LDR	First terminal	5V
2	LDR	Second terminal	A0 and one terminal of 10 kΩ resistor
3	10 kΩ resistor	Other terminal	GND
4	LED	Anode (+), long leg	Digital pin 8 through 220 Ω resistor
5	LED	Cathode (−), short leg	GND
6	Arduino Uno	USB port	Computer with Arduino IDE
7	Arduino Uno	GND	Common ground for the circuit

Conclusion:

Program

```
const int LDR_PIN = A0;
const int LED_PIN = 8;

String command = "";

void setup() {
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);


  Serial.begin(9600);

  Serial.println("==================================");
  Serial.println(" CPS MONITORING AND CONTROL UI");
  Serial.println("==================================");
  Serial.println("Commands:");
  Serial.println("ON     - Turn LED ON");
  Serial.println("OFF    - Turn LED OFF");
  Serial.println("STATUS - View system status");
  Serial.println("HELP   - Display commands");
  Serial.println("--------------------------------");
}

void loop() {
  // Read sensor data
  int lightValue = analogRead(LDR_PIN);


  // Monitor the sensor continuously
  Serial.print("Light Sensor Value: ");
  Serial.print(lightValue);
  Serial.print(" | LED Status: ");
  Serial.println(digitalRead(LED_PIN) ? "ON" : "OFF");
```

```
  // Receive commands from the user interface
  if (Serial.available() > 0) {
    command = Serial.readStringUntil('\n');
    command.trim();
    command.toUpperCase();

    if (command == "ON") {
      digitalWrite(LED_PIN, HIGH);
      Serial.println("SUCCESS: LED turned ON");
    }
    else if (command == "OFF") {
      digitalWrite(LED_PIN, LOW);
      Serial.println("SUCCESS: LED turned OFF");
    }
    else if (command == "STATUS") {
      Serial.println("----- SYSTEM STATUS -----");
      Serial.print("Light Value: ");
      Serial.println(analogRead(LDR_PIN));
      Serial.print("LED: ");
      Serial.println(digitalRead(LED_PIN) ? "ON" : "OFF");
      Serial.println("-------------------------");
    }
    else if (command == "HELP") {
      Serial.println("Enter ON, OFF, STATUS or HELP");
    }
    else {
      Serial.println("Invalid command. Type HELP.");
    }
  }

  delay(1000);
}
```

Experiment No. 5

Title: To implement security measures in an IIoT system using Arduino Uno

Aim: To implement security measures in an IIoT system using Arduino Uno and an HC-05 Bluetooth module, allowing password-based authentication and secure LED ON/OFF control through a mobile Bluetooth keypad, with status messages displayed on the Serial Monitor.

Components Required

Sr. No.	Component	Quantity
1	Arduino Uno	1
2	HC-05 Bluetooth module	1
3	LED	1
4	220 Ω resistor	1
5	Resistors for voltage divider (e.g., 1 kΩ and 2 kΩ)	1 each
6	Breadboard	1
7	Jumper wires	As required
8	USB cable	1
9	Computer with Arduino IDE	1
10	Android mobile phone with Bluetooth terminal app	1

Connection Table

Sr. No	Component	Pin/Terminal	Arduino Uno Connection
1	HC-05	VCC	5V
2	HC-05	GND	GND
3	HC-05	TXD	Digital pin 10 (Software RX)
4	HC-05	RXD	Digital pin 11 (Software TX) through a volta divider
5	LED	Anode (+), long leg	Digital pin 8 through a 220 Ω resistor
6	LED	Cathode (−), short le	GND
7	Arduino Uno	USB port	Computer running Arduino IDE

Theory:

The Industrial Internet of Things (IIoT) connects industrial devices, sensors, controllers, and monitoring systems to enable remote monitoring and control. Security is essential in IIoT because unauthorized access can lead to equipment misuse, production interruptions, or safety risks.

In this experiment, Arduino Uno acts as the controller, and the HC-05 Bluetooth module provides wireless communication between the Arduino and a mobile phone. The mobile Bluetooth terminal app acts as the user interface for entering the password and sending control commands.

The system uses password-based authentication. The user enters a four-digit password through the mobile keypad. If the password matches the predefined password (1234), access is granted, and the user can control the LED using the following commands:

- 1 — Turns the LED ON.
- 0 — Turns the LED OFF.
- L — Logs out and locks the system.

If the user enters an incorrect password three consecutive times, the system temporarily locks for 10 seconds and generates a security alert. Messages such as access granted, access denied, LED status, and system lockout are displayed on both the mobile Bluetooth terminal and the Arduino IDE Serial Monitor.

Procedure

1. Connect the HC-05 Bluetooth module and LED to the Arduino Uno according to the connection table.
2. Open the Arduino IDE and write the Bluetooth-based password authentication program.
3. Select Arduino Uno under Tools → Board and select the appropriate COM port.
4. Upload the program to the Arduino Uno.
5. Open the Serial Monitor and set the baud rate to 9600.
6. Pair the mobile phone with the HC-05 Bluetooth module and connect using a Bluetooth terminal app.
7. Enter the password 1234 using the mobile keypad. The system automatically verifies the password after four digits.
8. If authentication is successful, enter 1 to turn ON the LED and 0 to turn OFF the LED.
9. Enter L to log out and lock the system.

10. Enter an incorrect password three times consecutively and observe the security alert and 10-second lockout.
11. Observe the authentication messages, LED status, and security alerts on both the mobile Bluetooth terminal and the Arduino IDE Serial Monitor.
12. Record the observations and verify that the system allows LED control only after successful authentication.

Conclusion

The Bluetooth-based IIoT security system was successfully implemented using Arduino Uno and the HC-05 Bluetooth module. Password authentication was used to restrict unauthorized access, and the LED was controlled wirelessly through a mobile Bluetooth keypad. The system also detected incorrect password attempts, temporarily locked access after three consecutive failures, and displayed security and device-status messages on the Serial Monitor and mobile terminal. This experiment demonstrates the basic principles of authentication and access control in IIoT applications.

Program:

```
#include <SoftwareSerial.h>

// HC-05 connections
#define BT_RX 10
#define BT_TX 11

#define LED_PIN 8

SoftwareSerial BT(BT_RX, BT_TX);

// Demonstration password
const String PASSWORD = "2468";

String inputData = "";
bool authenticated = false;
int failedAttempts = 0;

const int MAX_ATTEMPTS = 3;
bool systemLocked = false;

void setup() {
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  Serial.begin(9600);
  BT.begin(9600);

  Serial.println("==================================");
  Serial.println(" IIoT SECURITY CONTROL SYSTEM");
  Serial.println("==================================");
  Serial.println("Connect mobile to HC-05.");
  Serial.println("Enter password followed by #.");
  Serial.println("Example: 2468#");
  Serial.println("LED is initially OFF.");
}

void loop() {
  while (BT.available() > 0) {
    char ch = BT.read();
```

```
    // Ignore line endings from mobile apps
    if (ch == '\r' || ch == '\n') {
      continue;
    }

    // # marks the end of a keypad command
    if (ch == '#') {
      inputData.trim();
      processCommand(inputData);
      inputData = "";
    }
    else if (inputData.length() < 30) {
      inputData += ch;
    }
    else {
      inputData = "";
      Serial.println("ERROR: Input too long.");
    }
  }
}

void processCommand(String cmd) {
  cmd.trim();

  Serial.print("Bluetooth Input: ");
  Serial.println(cmd);

  if (systemLocked) {
    Serial.println("ACCESS DENIED: System locked.");
    BT.println("SYSTEM LOCKED");
    return;
  }

  // Authentication stage
  if (!authenticated) {
    if (cmd == PASSWORD) {
      authenticated = true;
      failedAttempts = 0;

      Serial.println("Authentication successful.");
      Serial.println("Access granted.");
```

```
    Serial.println("Commands: ON#, OFF#, STATUS#, LOGOUT#");

    BT.println("AUTHENTICATION SUCCESS");
    BT.println("ACCESS GRANTED");
    BT.println("Use ON#, OFF#, STATUS#, LOGOUT#");
  }
  else {
    failedAttempts++;

    Serial.println("Authentication failed!");
    Serial.print("Failed attempts: ");
    Serial.println(failedAttempts);

    BT.println("WRONG PASSWORD");

    if (failedAttempts >= MAX_ATTEMPTS) {
      systemLocked = true;
      digitalWrite(LED_PIN, LOW);

      Serial.println("SECURITY ALERT: SYSTEM LOCKED.");
      Serial.println("Reset Arduino to restart.");

      BT.println("SYSTEM LOCKED");
    }
    else {
      BT.println("TRY AGAIN");
    }
  }

  return;
}

// Commands available after authentication
if (cmd == "ON") {
  digitalWrite(LED_PIN, HIGH);

  Serial.println("LED switched ON.");
  BT.println("LED ON");
}
else if (cmd == "OFF") {
```

```
    digitalWrite(LED_PIN, LOW);

    Serial.println("LED switched OFF.");
    BT.println("LED OFF");
  }
  else if (cmd == "STATUS") {
    Serial.print("Authentication: ");
    Serial.println(authenticated ? "GRANTED" : "DENIED");

    Serial.print("LED Status: ");
    Serial.println(digitalRead(LED_PIN) ? "ON" : "OFF");

    BT.print("LED STATUS: ");
    BT.println(digitalRead(LED_PIN) ? "ON" : "OFF");
  }
  else if (cmd == "LOGOUT") {
    authenticated = false;
    digitalWrite(LED_PIN, LOW);

    Serial.println("User logged out.");
    Serial.println("LED switched OFF for safety.");

    BT.println("LOGGED OUT");
    BT.println("LED OFF");
  }
  else {
    Serial.println("Invalid command.");
    BT.println("INVALID COMMAND");
  }
}
```